

Τεχνική Αναφορά Υλοποίησης

Μοντέλα Μηχανικής Μάθησης, Εκπαίδευση
και Ροή Δεδομένων μεταξύ των Κόμβων

Ανίχνευση και Αντιμετώπιση Επιθέσεων σε Δίκτυα
Οριζόμενα από Λογισμικό (SDN)

Φοιτήτρια: Ζαφειρώ Μπάρδα

Ημερομηνία: 9 Ιουλίου 2026

Αποθετήριο: github.com/ZafeiroBarda/ptyxiaki

Σκοπός του εγγράφου: τεκμηρίωση του τρόπου με τον οποίο εκπαιδεύονται και χρησιμοποιούνται τα μοντέλα ML, πού βρίσκονται στον κώδικα, και πώς μεταφέρονται τα δεδομένα ανάμεσα στους κόμβους και μέσα/έξω από το Docker.

Περιεχόμενα

1. Επισκόπηση αρχιτεκτονικής — δύο υλοποιήσεις, τέσσερα επίπεδα
2. Τα μοντέλα Μηχανικής Μάθησης: τι, πού, με ποια δεδομένα
3. Πώς εκπαιδεύεται κάθε μοντέλο (δεδομένα, χαρακτηριστικά, παράμετροι)
4. Πώς φορτώνονται και χρησιμοποιούνται τα μοντέλα στην inference
5. Ροή δεδομένων ανάμεσα στους κόμβους (Data $\leftrightarrow$ Control $\leftrightarrow$ Defense $\leftrightarrow$ Management)
6. Μεταφορά δεδομένων μέσα/έξω από το Docker
7. Ασφάλεια της ροής δεδομένων (control-plane)
8. Σύνοψη: το πλήρες μονοπάτι ενός πακέτου end-to-end
9. Αξιολόγηση: πώς τρέχουν τα tests & από πού προκύπτουν τα υψηλά νούμερα
10. Παράρτημα: χάρτης αρχείων

1. Επισκόπηση αρχιτεκτονικής

Το έργο περιέχει δύο συμπληρωματικές υλοποιήσεις που μοιράζονται τον ίδιο πυρήνα ανίχνευσης:

- **Υλοποίηση A** — η κύρια αρχιτεκτονική της διπλωματικής — ένα application-level SDN («Digital Twin») σε microservices Docker/Flask, με μη-επιβλεπόμενη ανίχνευση ανωμαλιών (Isolation Forest) και real-time dashboard (φάκελος app_sdn/).
- **Υλοποίηση B** — μια ακαδημαϊκή σύγκριση επιβλεπόμενης έναντι μη-επιβλεπόμενης μάθησης σε δεδομένα ροών (φάκελος ml_pipeline/), καθώς και ένας κλασικός SDN controller Mininet/Ryu με live ανίχνευση & αυτόματη αντιμετώπιση (φάκελοι simulation/ και controller/).

Και οι δύο οργανώνονται στα ίδια τέσσερα λογικά επίπεδα (planes) του SDN:

Επίπεδο (Plane)	Ρόλος	Αρχείο υλοποίησης
Data Plane	Προώθηση πακέτων· τοπική συλλογή στατιστικών ανά πηγή IP	app_sdn/switch.py
Control Plane	Global Network View + Flow Table + REST API (southbound)	app_sdn/controller.py
Intelligence & Defense	Isolation Forest: απόφαση Normal/Attack + αυτόματο DROP	app_sdn/controller.py (DefenseEngine)
Management Plane	Real-time οπτικοποίηση, χειροκίνητος έλεγχος επιθέσεων	app_sdn/dashboard.py
Orchestration	Δικτύωση & εκκίνηση όλων των υπηρεσιών	app_sdn/docker-compose.yml

Το κλειδί που ενοποιεί τα πάντα είναι η κοινή μηχανή ανίχνευσης *ml_pipeline/detection_engine.py*: η ίδια λογική εξαγωγής χαρακτηριστικών και ταξινόμησης χρησιμοποιείται από τον *Flask controller*, τον *Ryu controller* και τον *offline προσομοιωτή* (αρχή DRY).

2. Τα μοντέλα Μηχανικής Μάθησης: τι, πού, με ποια δεδομένα

Στο σύστημα συνυπάρχουν τέσσερις διακριτές «οικογένειες» μοντέλων. Δύο δουλεύουν offline στα πλούσια χαρακτηριστικά ροής (24 features τύπου CICFlowMeter) και δύο δουλεύουν live στα 8 συγκεντρωτικά (aggregate) χαρακτηριστικά που μπορεί πραγματικά να υπολογίσει ένας SDN controller σε πραγματικό χρόνο. Ο παρακάτω πίνακας τα συνοψίζει:

Μοντέλο(α)	Είδος	Αρχείο εκπαίδευσης	Artifact (.pkl)
6 επιβλεπόμενα (RF, DT, KNN, SVM, LogReg, MLP)	Supervised, 24 features	ml_pipeline/train.py	models/best_model.pkl
Isolation Forest (offline)	Unsupervised, 24 features	ml_pipeline/isolation_forest_detector.py	models/isolation_forest.pkl
Isolation Forest (live)	Unsupervised, 8 aggregate	app_sdn/train_defense_engine.py	models/isolation_forest_live.pkl
Random Forest (live)	Supervised, 8 aggregate	controller/train_live_model.py	models/live_model.pkl

Γιατί δύο διαφορετικά σύνολα χαρακτηριστικών;

Το OpenFlow δίνει ανά ροή μόνο τρία μεγέθη: packet_count, byte_count, duration (config.py:59-61). Επομένως ο live controller ΔΕΝ μπορεί να υπολογίσει τα 24 πλούσια CICFlowMeter features· αντ' αυτού συγκεντρώνει στατιστικά ανά πηγή IP σε κάθε παράθυρο δειγματοληψίας. Έτσι τα offline μοντέλα (για τα τελικά, ακαδημαϊκά αποτελέσματα) και τα live μοντέλα (για τη λειτουργία σε πραγματικό χρόνο) εκπαιδεύονται σκόπιμα σε διαφορετικά feature sets.

2.1 Τα δύο σύνολα χαρακτηριστικών

Offline — 24 features (config.FEATURE_COLUMNS, config.py:31-56): Flow_Duration, Tot_Fwd_Pkts, Tot_Bwd_Pkts, Flow_Byts_s, Flow_Pkts_s, Flow_IAT_Mean/Std, SYN/ACK/FIN/RST_Flag_Cnt, Pkt_Len_Mean/Std, Down_Up_Ratio, Init_Fwd_Win_Byts κ.ά. — ίδια δομή με το InSDN.

Live — 8 aggregate features (config.LIVE_FEATURE_COLUMNS, config.py:63-72): flow_count, total_packets, total_bytes, avg_packets_per_flow, avg_bytes_per_flow, avg_duration, avg_pkt_size, short_flow_ratio.

Το short_flow_ratio (ποσοστό ροών με ≤3 πακέτα) είναι το πιο διακριτικό χαρακτηριστικό για flood/scan: μια επίθεση παράγει πολλές μικροσκοπικές ροές, ενώ ένα νόμιμο download έχει λίγες ροές με πολλά πακέτα.

3. Πώς εκπαιδεύεται κάθε μοντέλο

3.1 Επιβλεπόμενα μοντέλα offline — ml_pipeline/train.py

Κοινή διαδικασία (train.py:84-147) για τα έξι μοντέλα που ορίζονται στη build_models() (train.py:44-66):

11. Φόρτωση δεδομένων: συνθετικό CSV ή πραγματικό InSDN, μέσω preprocess.load_synthetic() / load_insdn().
12. Καθαρισμός: αντικατάσταση $\pm inf \rightarrow NaN$, αφαίρεση NaN. Για το InSDN ΔΕΝ αφαιρούνται διπλότυπα (dedup=False), γιατί τα flood παράγουν νόμιμα πανομοιότυπες ροές — η αφαίρεσή τους θα έσβηνε σχεδόν όλη την κλάση DDoS (preprocess.py:26-37, 83).
13. Διαχωρισμός: 70% εκπαίδευση / 30% έλεγχος, στρωματοποιημένος ώστε να διατηρείται η αναλογία κλάσεων (TEST_SIZE=0.30, RANDOM_STATE=42, preprocess.py:111-116).
14. Κανονικοποίηση: StandardScaler — fit MONO στο train, transform στο test (αποφυγή data leakage, preprocess.py:118-122).
15. Εκπαίδευση & αξιολόγηση καθενός· μετρικές macro-averaged (accuracy/precision/recall/F1, train.py:69-81).
16. Αποθήκευση του καλύτερου (κατά F1) ως models/best_model.pkl μαζί με scaler.pkl, label_encoder.pkl, feature_columns.json, meta.json (train.py:135-147).

Οι υπερπαράμετροι των μοντέλων (train.py:46-65):

```
RandomForestClassifier(n_estimators=120, max_depth=None, n_jobs=-1)
DecisionTreeClassifier(max_depth=20)
KNeighborsClassifier(n_neighbors=5)
SVC(kernel='rbf', C=10, gamma='scale')
LogisticRegression(max_iter=1000)
MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=300)
```

Τα έξι επιβλεπόμενα μοντέλα, με σταθερό random_state=42 για αναπαραγωγικότητα.

Εκτέλεση: python3 ml_pipeline/train.py (συνθετικά) ή python3 ml_pipeline/train.py --insdn (πραγματικό InSDN, train.py:156-161).

3.2 Isolation Forest offline (μη-επιβλεπόμενο) — isolation_forest_detector.py

Η βασική ιδέα (isolation_forest_detector.py:9-13): το μοντέλο εκπαιδεύεται MONO σε φυσιολογική κίνηση, χωρίς ετικέτες επιθέσεων. Στην inference, όταν η συνάρτηση απόφασης $f(x) < 0$ το δείγμα θεωρείται outlier (ύποπτο).

17. Το πρόβλημα γίνεται δυαδικό: Normal (0) vs οποιαδήποτε επίθεση (1) — load_binary(), γραμμές 49-59.
18. Κρατείται το 60% της φυσιολογικής κίνησης για εκπαίδευση· το test set = υπόλοιπη φυσιολογική + ΟΛΕΣ οι επιθέσεις (γραμμές 80-91).
19. Λογαριθμικός μετασχηματισμός log1p σε όλα τα features (γραμμές 62-69, 93-95). Τα network features είναι log-normal· χωρίς αυτόν το ROC AUC πέφτει από ~0.93 σε ~0.70.
20. StandardScaler με fit MONO στα φυσιολογικά train (γραμμές 97-100).

```
IsolationForest(n_estimators=300, max_samples=512, contamination=0.05,  
                max_features=0.5, random_state=42, n_jobs=-1)  
raw = iso.predict(X_test)      # +1 = inlier, -1 = outlier  
y_pred = (raw == -1)          # 1 = επίθεση
```

Βελτιστοποιημένες παράμετροι (isolation_forest_detector.py:105-113).

Έξοδος: models/isolation_forest.pkl + scaler + μετρικές/γραφήματα στο results/ (γραμμές 131-162).

3.3 Isolation Forest live — app_sdn/train_defense_engine.py

Αυτό είναι το μοντέλο που τρέχει ΖΩΝΤΑΝΑ μέσα στον Flask controller. Εκπαιδεύεται στα 8 aggregate features — ακριβώς αυτά που στέλνει το Data Plane ως τηλεμετρία — και πάλι ΜΟΝΟ σε φυσιολογική κίνηση (unsupervised):

21. Παράγονται 20.000 συνθετικά «φυσιολογικά» aggregate δείγματα με ρεαλιστικό μέγεθος πακέτου (~250-1400 bytes) και λίγες σύντομες ροές (gen_normal_aggregate(), train_defense_engine.py:32-54).
22. StandardScaler + IsolationForest(n_estimators=300, max_samples=512, max_features=0.75, contamination=0.03) — χαμηλό contamination ώστε το live σύστημα να παράγει ελάχιστα false positives (γραμμές 62-75).
23. Έξοδος: models/isolation_forest_live.pkl + isolation_forest_live_scaler.pkl (γραμμές 77-79).

3.4 Random Forest live — controller/train_live_model.py

Εφεδρικό/εναλλακτικό live μοντέλο (επιβλεπόμενο) για τον Ryu/POX controller. Εκπαιδεύεται επίσης στα 8 aggregate features, σε συνθετικά δεδομένα με ετικέτες Normal/Attack (train_live_model.py:36-99): RandomForest 80 δέντρων, 70/30 split, StandardScaler. Έξοδος: models/live_model.pkl + live_scaler.pkl. Μπορεί να ξαναεκπαιδευτεί στα δικά σου δεδομένα με --collected.

3.5 Αποτελέσματα εκπαίδευσης (πραγματικό InSDN, 343.516 ροές)

Τα επιβλεπόμενα μοντέλα, εκπαιδευμένα στο πλήρες πραγματικό InSDN
(results/model_comparison_insdn.csv):

Μοντέλο	Accuracy	Precision	Recall	F1 (macro)
KNN	0.990	0.979	0.932	0.952
Random Forest	0.990	0.961	0.931	0.945
Decision Tree	0.989	0.952	0.936	0.943
MLP (Neural Net)	0.974	0.969	0.905	0.931
SVM (RBF)	0.939	0.941	0.808	0.832
Logistic Regression	0.911	0.825	0.731	0.729

Μη-επιβλεπόμενο Isolation Forest στο ίδιο πλήρες InSDN

(results/isolation_forest_metrics_insdn.csv): **Precision 0.993, Recall 0.676, F1 0.804, ROC-AUC 0.930**. Η πτώση του recall σε σχέση με το συνθετικό (0.99) είναι αναμενόμενη και τεκμηριώνει γιατί το επιβλεπόμενο/υβριδικό σχήμα υπερτερεί όταν υπάρχουν ετικέτες.

4. Πώς φορτώνονται & χρησιμοποιούνται τα μοντέλα στην inference

Τα μοντέλα δεν εκπαιδεύονται κατά την εκτέλεση: φορτώνονται έτοιμα από τα .pkl (serialization με joblib). Στην κύρια αρχιτεκτονική:

24. Κατά την εκκίνηση του Flask controller φορτώνεται το live Isolation Forest μία φορά, σε επίπεδο module (controller.py:52-60). Αν αποτύχει, το σύστημα πέφτει σε εφεδρική λογική αντί να σταματήσει.

```
_ISO = joblib.load('models/isolation_forest_live.pkl')
_ISO_SCALER = joblib.load('models/isolation_forest_live_scaler.pkl')
```

Φόρτωση του live μοντέλου κατά το import (app_sdn/controller.py:54-56).

25. Κάθε αίτημα τηλεμετρίας περνά από την DefenseEngine.analyze() (controller.py:138-153), που ακολουθεί αλυσίδα προτεραιότητας:

- 1η επιλογή: Isolation Forest — iso.predict(X)==-1 ⇒ 'Attack'.
- Εφεδρεία: engine.classify() με το live Random Forest (live_model.pkl).
- Τελευταία εφεδρεία: ευριστικός κανόνας (πολλές σύντομες ροές = flood), detection_engine.py:53-62.

Πριν την ταξινόμηση, τα ακατέργαστα flows μετατρέπονται στα 8 aggregate features από την κοινή engine.aggregate_features() (detection_engine.py:33-50) — η ΙΔΙΑ συνάρτηση που θα χρησιμοποιούσε και ο Ryu controller. Στην Υλοποίηση Β, ο Ryu controller φορτώνει αντίστοιχα το live_model.pkl (detection_controller.py:67-76) και εφαρμόζει την ίδια λογική στα flow-stats του OpenFlow.

5. Ροή δεδομένων ανάμεσα στους κόμβους

Η επικοινωνία των κόμβων γίνεται με REST/JSON πάνω από HTTP (southbound API που προσομοιώνει το OpenFlow). Δεν υπάρχει κοινή μνήμη — κάθε κόμβος στέλνει και λαμβάνει μηνύματα.

5.1 Data Plane → Control Plane: τηλεμετρία

Ο switch (app_sdn/switch.py) συγκεντρώνει στατιστικά ανά πηγή IP και ανά προορισμό (μία ροή = ένας προορισμός, switch.py:57-81), και τα στέλνει περιοδικά (κάθε 3s) στον controller με POST /telemetry:

```
POST http://controller:9000/telemetry
Headers: X-Switch-Token: sdn-secret-2024
{
  "src": "10.0.0.6",
  "dst": "10.0.0.5",
  "flows": [[packets, bytes, duration], ...]
}
```

Το μήνυμα τηλεμετρίας (switch.py:83-101). Κάθε στοιχείο της flows είναι μία ροή.

Ο controller (controller.py:258-372) το επεξεργάζεται ως εξής:

26. Επικύρωση σχήματος + rate limiting + έλεγχος token (βλ. §7).
27. Ενημέρωση του Global Network View με τη ροή src→dst (update_flow).
28. DefenseEngine.analyze(flows) ⇒ verdict 'Attack'/'Normal' + το feature vector.
29. Αν 'Attack' και δεν υπάρχει ήδη κανόνας: εγκατάσταση DROP στο Flow Table με TTL 10s και σήμανση του κόμβου ως 'blocked' (controller.py:341-346).
30. Επιστροφή απόφασης στον switch: { action: FORWARD | DROP, verdict, anomaly_score }.

Ο switch διαβάζει την απάντηση: αν action == DROP, προσθέτει την πηγή στο τοπικό σύνολο blocked και εφαρμόζει την απόρριψη στο Data Plane — δηλαδή η εντολή του Control Plane επιβάλλεται στον κόμβο (switch.py:95-99, 64-66).

5.2 Control/Defense → Data Plane: αντιμετώπιση (mitigation)

Ο κανόνας DROP ζει στο FlowTable (controller.py:93-123): ένα λεξικό src_ip → {action, priority, expires}. Οι κανόνες λήγουν αυτόματα με τον χρόνο (action_for/expire), οπότε ο αποκλεισμός είναι προσωρινός και το σύστημα επανέρχεται μόνο του. Στην Υλοποίηση Β με πραγματικό OpenFlow, το ίδιο γίνεται με ένα OFPFlowMod υψηλής προτεραιότητας και κενές actions = DROP (detection_controller.py:221-229, ryu_bridge.py:196-199).

5.3 Management Plane: το dashboard τραβάει (pull) την κατάσταση

Το dashboard (Dash/Plotly) δεν συνδέεται στη ροή· κάνει poll κάθε 1 δευτερόλεπτο (dcc.Interval, dashboard.py:185) και τραβάει την κατάσταση από τέσσερα endpoints του controller: /stats, /metrics, /flow_table, /anomaly_scores (dashboard.py:213-219). Στέλνει επίσης χειροκίνητες εντολές επίθεσης μέσω POST /simulate/command, τις οποίες η προσομοίωση διαβάζει με GET /simulate/poll.

5.4 Το πλήρες southbound/northbound API

Endpoint	Μέθοδος	Ρόλος στη ροή δεδομένων
/register	POST	Εγγραφή κόμβου στο Global Network View· καταχώρηση trusted switch
/telemetry	POST	Ο switch στέλνει στατιστικά ροών· ο controller αποφασίζει
/flow_table	GET	Τρέχοντες κανόνες FORWARD/DROP
/topology	GET	Global Network View (κόμβοι + ενεργές ροές)
/stats, /metrics	GET	Μετρικές ανίχνευσης & ιστορικό ρυθμού πακέτων
/anomaly_scores	GET	Τελευταίο IF anomaly score ανά host IP
/unblock, /reset	POST	Άρση αποκλεισμών / μηδενισμός (admin σε DEFENSE_MODE)
/simulate/command-poll	POST·GET	Χειροκίνητος έλεγχος επιθέσεων από το dashboard
/health	GET	Έλεγχος υγείας (healthcheck του Docker)

6. Μεταφορά δεδομένων μέσα/έξω από το Docker

6.1 Δικτύωση μεταξύ των containers

Όλες οι υπηρεσίες προσαρτώνται σε ένα κοινό bridge network, το `sdn_net` (docker-compose.yml:128-130). Χάρη στο ενσωματωμένο DNS του Docker, κάθε container βρίσκει τα άλλα με το ΟΝΟΜΑ της υπηρεσίας — γι' αυτό ο switch και το dashboard μιλούν στη διεύθυνση `http://controller:9000` χωρίς σκληροκωδικοποιημένες IP (μεταβλητή `CONTROLLER_URL`).

Υπηρεσία	Ρόλος	Θύρα (host→container)	Πώς επικοινωνεί
controller	Control + Defense Plane	9000 → 9000	δέχεται REST από switch/mininet
dashboard	Management Plane (Web UI)	8050 → 8050	GET/POST προς controller:9000
switch	Data Plane (UDP overlay)	— (μόνο εσωτερικά)	POST telemetry → controller
mininet	Πραγματική τοπολογία OVS	— (privileged)	ovs-ofctl → REST → controller
ryu	OpenFlow controller (profile)	6653 OFP	OpenFlow ↔ OVS, REST → controller

Μόνο δύο θύρες δημοσιεύονται προς τον host: 9000 (REST API) και 8050 (το dashboard στο `http://localhost:8050`). Οι υπόλοιπες υπηρεσίες μιλούν αποκλειστικά εσωτερικά μέσα στο `sdn_net`.

6.2 Πώς φτάνει ο κώδικας & τα μοντέλα στα containers

Δεν χρησιμοποιούνται volumes για τον κώδικα: το `app_sdn/Dockerfile` αντιγράφει (COPY) μέσα στο image τους φακέλους `ml_pipeline/`, `controller/`, `app_sdn/` και — κρίσιμα — `models/` (Dockerfile:11-14). Έτσι τα εκπαιδευμένα .pkl «ψήνονται» μέσα στο image κατά το build· όταν ξεκινά ο controller, φορτώνει το `isolation_forest_live.pkl` από τον τοπικό του δίσκο. Συνέπεια: αν ξαναεκπαιδεύσεις ένα μοντέλο, χρειάζεται rebuild (`docker compose up --build`) για να μπει η νέα έκδοση στο image.

6.3 Ρύθμιση μέσω μεταβλητών περιβάλλοντος

Η συμπεριφορά κάθε container ρυθμίζεται με environment variables στο docker-compose.yml — όχι με αλλαγές κώδικα:

- `SWITCH_API_KEY` — το κοινό μυστικό token switch↔controller (default `sdn-secret-2024`).
- `DEFENSE_MODE` — 0/1· στο 1 ενεργοποιεί τον αποκλεισμό μη-εξουσιοδοτημένης τηλεμετρίας.
- `CONTROLLER_URL` — πού βρίσκουν τον controller το dashboard/mininet.
- `NORMAL_PHASE` / `ATTACK_PHASE` / `RECOVERY_PHASE` / `LOOP_CYCLES` — χρονισμός του σεναρίου demo.

6.4 Ο κόμβος Mininet μέσα στο Docker

Το `simulation/Dockerfile.mininet` φτιάχνει έναν Ubuntu container με Mininet + Open vSwitch, που τρέχει σε privileged mode με `cap NET_ADMIN/SYS_ADMIN` — απαραίτητα για τα network namespaces του Mininet (`docker-compose.yml:73-76`). Ο OVS λειτουργεί σε standalone (self-learning L2) mode με userspace datapath, ώστε να δουλεύει και σε WSL2 χωρίς kernel module (`mininet_live.py:283`). Τα στατιστικά διαβάζονται με `ovs-ofctl dump-ports` (`mininet_live.py:100-117`) και προωθούνται ως REST τηλεμετρία στον controller μέσω του `sdn_net`.

6.5 Πώς βγαίνουν τα δεδομένα ΕΞΩ από το Docker

Υπάρχουν τρεις δρόμοι εξαγωγής:

31. Ζωντανό CSV: ο controller γράφει κάθε απόφαση σε `data/live_mininet_flows.csv` (14 στήλες: `timestamp, src/dst, packets, bytes, verdict, action, anomaly_score, ...`· `controller.py:186-210, 349-369`) μέσα στο δικό του filesystem.
32. REST export: το dashboard εκθέτει `/export/flows.csv` και `/export/blocked.csv`, που παράγουν CSV on-the-fly από τα `/metrics` και `/flow_table` του controller (`dashboard.py:525-561`).
33. Το demo script (`demo_10min.sh:220-221`) κατεβάζει αυτά τα CSV με curl και τα αποθηκεύει τοπικά στο `results/live/<run_id>` μαζί με ένα `summary.json`.

Παράδειγμα αποτελέσματος demo (`results/live/demo_run_20260620_031742/summary.json`):

attacker_blocked=YES, detection_time=0s, false_positives=0, total_detections=34. Για να πάρεις οποιοδήποτε αρχείο απευθείας από ένα container: `docker compose cp sdn_controller:/app/data/live_mininet_flows.csv` .

7. Ασφάλεια της ροής δεδομένων (control-plane)

Επειδή η τηλεμετρία καθορίζει ποιος μπλοκάρεται, το κανάλι switch→controller προστατεύεται από injection επιθέσεις (ένας κακόβουλος να στείλει ψεύτικη τηλεμετρία που μπλοκάρει αθώο host). Οι άμυνες, όλες στο /telemetry:

- **Αυθεντικοποίηση** — Έλεγχος token: κάθε αίτημα πρέπει να φέρει X-Switch-Token ίσο με το SWITCH_API_KEY· αλλιώς καταγράφεται ως injection attempt (controller.py:293-309).
- **Αποκλεισμός** — Σε DEFENSE_MODE=1, αιτήματα χωρίς έγκυρο token απορρίπτονται με HTTP 401 πριν καν αναλυθούν.
- **Rate limiting** — Το πολύ RATE_LIMIT_MAX (120) αιτήματα ανά IP ανά 60s· υπέρβαση ⇒ HTTP 429 (controller.py:217-226, 282-285).
- **Επικύρωση εισόδου** — Έλεγχος σχήματος JSON, μέγιστο 5000 ροές/αίτημα, μέγιστο μήκος πεδίων IP (controller.py:266-291).
- **Admin endpoints** — Τα /reset και /unblock απαιτούν X-Admin-Token σε DEFENSE_MODE (controller.py:434-447).

8. Σύνοψη: το πλήρες μονοπάτι ενός πακέτου (end-to-end)

34. Ένας host (π.χ. h6) παράγει κίνηση προς τον στόχο (h5). Στην Υλοποίηση A το «πακέτο» είναι UDP datagram προς τον switch· στη B είναι πραγματικό πακέτο που φτάνει στον OVS.
35. Ο κόμβος Data Plane το καταγράφει και αθροίζει στατιστικά ανά πηγή IP (switch.ingest / OVS dump-ports / OpenFlow PACKET_IN).
36. Ανά παράθυρο (3-5s) στέλνεται τηλεμετρία στον controller: POST /telemetry με X-Switch-Token.
37. Ο controller μετατρέπει τα flows σε 8 aggregate features (aggregate_features) και τρέχει το Isolation Forest live.
38. Αν $f(x) < 0 \Rightarrow \text{verdict 'Attack'}$: εγκαθίσταται κανόνας DROP στο Flow Table (TTL 10s) και ο κόμβος σημειώνεται 'blocked'.
39. Η απόφαση επιστρέφει στον κόμβο, που εφαρμόζει το DROP στο Data Plane (ή, στη B, εγκαθίσταται OFPFlowMod DROP στον OVS).
40. Το dashboard, κάνοντας poll κάθε 1s, δείχνει τον αποκλεισμό, το anomaly score και τον ρυθμό πακέτων σε πραγματικό χρόνο.
41. Όταν λήξει το TTL, ο κανόνας φεύγει αυτόματα και ο host επανέρχεται — εκτός αν η επίθεση συνεχίζεται, οπότε ξαναμπλοκάρεται στον επόμενο κύκλο.

Έτσι κλείνει ο βρόχος Data Collection → Model Inference → Automated Mitigation → Feedback, πλήρως αυτοματοποιημένος και χωρίς ανθρώπινη παρέμβαση.

9. Αξιολόγηση: πώς τρέχουν τα tests και από πού προκύπτουν τα υψηλά νούμερα

Η ενότητα αυτή εξηγεί δύο διαφορετικά πράγματα που συχνά μπερδεύονται: (α) τα αυτοματοποιημένα tests που ελέγχουν ΟΡΘΟΤΗΤΑ, και (β) την αξιολόγηση επίδοσης των μοντέλων (accuracy/precision/F1). Στη συνέχεια δίνεται τεκμηριωμένη εξήγηση για το γιατί οι μετρικές είναι τόσο υψηλές — ερώτημα που πρέπει να απαντηθεί με ειλικρίνεια στην υπεράσπιση.

9.1 Τα αυτοματοποιημένα tests (pytest) — τι ελέγχουν

Το `pytest tests/` τρέχει 79 tests σε 7 αρχεία. Επιβεβαιωμένο αποτέλεσμα αυτής της εκτέλεσης: **79 passed σε 27,94s**. Κρίσιμη διάκριση: τα tests ΔΕΝ ελέγχουν αν το accuracy είναι υψηλό — ελέγχουν αν ο κώδικας συμπεριφέρεται σωστά (σχήματα, endpoints, ασφάλεια, λογική ανίχνευσης).

Αρχείο (πλήθος)	Τι επαληθεύει
<code>test_pipeline.py</code> (11)	Σχήμα synthetic δεδομένων, split/scaling, aggregate_features, ευριστικός κανόνας
<code>test_api_controller.py</code> (19)	Όλα τα REST endpoints μέσω Flask test_client (register/telemetry/stats/...)
<code>test_security_validation.py</code> (13)	Token, DEFENSE_MODE 401, rate-limit 429, επικύρωση εισόδου
<code>test_model_regression.py</code> (10)	Φόρτωση μοντέλου, normal→+1 / attack→-1, inference <100ms, καθαρό scaling
<code>test_live_scenarios.py</code> (10)	Σενάρια επίθεσης (flood/scan) στη μηχανή ανίχνευσης
<code>test_docker_health.py</code> (9)	Υγεία/ρυθμίσεις των υπηρεσιών Docker
<code>test_app_sdn.py</code> (7)	End-to-end ροή Data→Control→Defense σε επίπεδο εφαρμογής

Παράδειγμα: το `test_attack_traffic_classified_attack` (`test_model_regression.py:75-82`) στέλνει 90 πολύ σύντομες ροές (μοτίβο SYN flood) και απαιτεί το Isolation Forest να επιστρέψει -1 (ανωμαλία). Είναι έλεγχος συμπεριφοράς σε ένα χειροποίητο, ακραίο δείγμα — όχι μέτρηση επίδοσης σε dataset.

9.2 Πώς προκύπτουν οι μετρικές επίδοσης

Χωριστά από τα tests, οι μετρικές παράγονται από το `train.py` / `evaluate.py`:

42. Stratified split 70/30 (RANDOM_STATE=42) — διατηρεί την αναλογία κλάσεων.
43. StandardScaler με fit MONO στο train (αποφυγή leakage από την κανονικοποίηση).
44. fit στο train, predict στο 30% test set που δεν είδε το μοντέλο.
45. Υπολογισμός macro-averaged accuracy/precision/recall/F1 (κάθε κλάση ίσο βάρος) → `results/model_comparison*.csv` (`train.py:69-132`).
46. `evaluate.py`: confusion matrix, feature importance, classification report ανά κλάση.

9.3 Από πού προκύπτουν τα ΥΨΗΛΑ νούμερα — τρεις διακριτές πηγές

Πηγή 1 — Συνθετικά δεδομένα $\approx 99,98\%$: τεχνητά «εύκολα».

Ο γεννήτορας (generate_synthetic_dataset.py) φτιάχνει κάθε κλάση από ΞΕΧΩΡΙΣΤΕΣ, μη-επικαλυπτόμενες κατανομές στα πιο διακριτικά features. Π.χ. το DDoS έχει Flow_Pkts_s ≈ 50.000 και πακέτα ~ 80 bytes, ενώ το Normal έχει μέτριο ρυθμό και πακέτα ~ 550 bytes (γραμμές 74-107 vs 38-71). Έτσι οι κλάσεις είναι σχεδόν τέλεια διαχωρίσιμες και κάθε ταξινομητής πιάνει $\sim 100\%$. Το ίδιο το αρχείο προσφέρει σημαία --hard που προσθέτει 15% θόρυβο + 8% επικάλυψη κλάσεων και ρίχνει το accuracy σε ρεαλιστικά $\sim 0,90-0,97$ (γραμμές 227-255). Το αποθηκευμένο CSV είναι σε easy mode· γι' αυτό βλέπουμε 99,98%. Συμπέρασμα: το νούμερο αυτό είναι artifact της γεννήτριας — χρησιμεύει μόνο για ανάπτυξη του pipeline, όπως δηλώνει ρητά και το README.

Πηγή 2 — Πραγματικό InSDN accuracy $\approx 99\%$: φουσκώνει από διπλότυπα + ανισορροπία.

Μετρήσαμε στο πλήρες InSDN ότι **το 51,7% των γραμμών είναι ΑΚΡΙΒΩΣ διπλότυπα διανύσματα χαρακτηριστικών**. Η κατανομή ανά κλάση είναι αποκαλυπτική:

Κλάση	Ποσοστό δείγματος	Διπλότυπα (ίδια features)
DDoS	35,5% (121.942)	99,3%
Probe	28,6% (98.129)	45,7%
Normal	19,9% (68.424)	8,7%
DoS	15,6% (53.616)	8,9%
BFA	0,4% (1.405)	3,6%

Το preprocess καλεί σκόπιμα dedup=False για το InSDN (preprocess.py:83), γιατί τα flood ΠΑΡΑΓΟΥΝ νόμιμα όμοιες ροές. Όμως σε συνδυασμό με τυχαίο split, σχεδόν κάθε test ροή DDoS έχει ΠΑΝΟΜΟΙΟΤΥΠΟ δίδυμο μέσα στο train set — άρα ο ταξινομητής σε μεγάλο βαθμό «θυμάται» αντί να γενικεύει (μορφή train/test leakage). Επειδή το DDoS+Probe+Normal είναι το 84% των δεδομένων, το accuracy κυριαρχείται από αυτές τις εύκολες κλάσεις και ανεβαίνει στο $\sim 99\%$.

Πηγή 3 — Το TIMIO νούμερο είναι το macro-F1 = 0,945 (και IF ROC-AUC 0,930).

Το macro-F1 δίνει ίσο βάρος σε ΚΑΘΕ κλάση, οπότε αποκαλύπτει τη δύσκολη μειοψηφική BFA ($F1 \approx 0,71$ με μόλις 1.405 δείγματα). Γι' αυτό ενώ το accuracy είναι $\sim 0,99$, το macro-F1 πέφτει στο 0,945 — αυτή η διαφορά ΕΙΝΑΙ το μέτρο της πραγματικής δυσκολίας. Αντίστοιχα, στο μη-επιβλεπόμενο Isolation Forest το recall πέφτει από 0,99 (συνθετικά) σε 0,676 (πλήρες InSDN): η τίμια, χαμηλότερη εικόνα των πραγματικών δεδομένων.

9.4 Γιατί, παρ' όλα αυτά, το γνήσιο σήμα είναι όντως ισχυρό

Δεν είναι όλα τεχνητά: οι επιθέσεις flood/scan έχουν ΠΡΑΓΜΑΤΙΚΑ ακραία flow στατιστικά (ρυθμοί πακέτων τάξεις μεγέθους πάνω από το φυσιολογικό, ελάχιστα bytes/πακέτο), οπότε η

flow-based ανίχνευση δουλεύει γνήσια καλά. Η βιβλιογραφία αναφέρει τακτικά >99% accuracy στο InSDN. Το μήνυμα δεν είναι «τα μοντέλα κάνουν λάθος», αλλά «το headline accuracy υπερεκτιμά τη δυσκολία· το σωστό νούμερο αναφοράς είναι το macro-F1».

9.5 Τι να παρουσιαστεί στην υπεράσπιση (defense-ready)

- **Χρησιμοποίησε το σωστό μέτρο** — Ανάφερε ως κύριες μετρικές το macro-F1 (0,945) και το IF ROC-AUC (0,930) — όχι το accuracy, που είναι παραπλανητικό σε ανισόρροπα δεδομένα.
- **Ξεκαθάρισε τα συνθετικά** — Δήλωσε ρητά ότι τα συνθετικά (99,98%) είναι για ανάπτυξη του pipeline, όχι τελικό αποτέλεσμα.
- **Παραδέξου τον περιορισμό** — Ανάφερε τα διπλότυπα (DDoS 99,3%) ως γνωστό περιορισμό των CIC/InSDN datasets και το leakage που εισάγει το τυχαίο split.
- **Πρότεινε μελλοντική δουλειά** — Πρότεινε ως βελτίωση flow-grouped ή temporal split (ή deduplication πριν το split), που δίνει πιο συντηρητικά, πιο τίμια νούμερα — και εξηγεί γιατί η δύσκολη κλάση BFA παραμένει ανοιχτό ζήτημα.

10. Παράρτημα: χάρτης αρχείων

Αρχείο	Τι κάνει
<code>ml_pipeline/config.py</code>	Κεντρικές ρυθμίσεις: 24 offline + 8 live features, κατώφλια, seeds
<code>ml_pipeline/preprocess.py</code>	Φόρτωση/καθαρισμός δεδομένων, split, scaling
<code>ml_pipeline/generate_synthetic_dataset.py</code>	Γεννήτορας συνθετικών (easy/--hard mode)
<code>ml_pipeline/train.py</code>	Εκπαίδευση & σύγκριση 6 επιβλεπόμενων μοντέλων
<code>ml_pipeline/evaluate.py</code>	Γραφήματα & classification report (μετρικές)
<code>ml_pipeline/isolation_forest_detector.py</code>	Offline unsupervised IF (24 features, log1p)
<code>ml_pipeline/detection_engine.py</code>	Κοινή μηχανή: aggregate_features + classify
<code>app_sdn/train_defense_engine.py</code>	Εκπαίδευση live IF (8 aggregate features)
<code>app_sdn/controller.py</code>	Control + Defense Plane, REST API, Flow Table
<code>app_sdn/switch.py</code>	Data Plane: συλλογή στατιστικών + αποστολή τηλεμετρίας
<code>app_sdn/dashboard.py</code>	Management Plane: real-time UI + CSV export
<code>app_sdn/docker-compose.yml</code>	Orchestration: sdn_net, θύρες, env vars
<code>controller/train_live_model.py</code>	Εκπαίδευση live RF (8 aggregate features)
<code>controller/detection_controller.py</code>	Ryu controller: live detection + OpenFlow DROP
<code>simulation/mininet_live.py</code>	Live demo Mininet+OVS, τηλεμετρία μέσω ovs-ofctl
<code>simulation/ryu_bridge.py</code>	Γέφυρα OpenFlow ↔ Flask controller
tests/ (7 αρχεία, 79 tests)	Αυτοματοποιημένοι έλεγχοι ορθότητας (pytest)

— Τέλος τεχνικής αναφοράς —